

1. Business Problem Understanding

The Project aims to predict whether a loan will be approved or not based on customer attributes. This is a binary classification problem, with the target variable being "loan_status"(values like "Approved" or "Not Approved"). This Prediction will help financial institution asses the risk associated with lending money to individuals or businesses, thereby enabling better decision-making.



Data Understanding

```
In [4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

```
In [5]: df=pd.read_csv(r"D:\CAPSTONE PROJECT_DEPLOYMENT\11. CAPSTONE PROJECT_DEPLOYMENT\
df.head()
```

Out[5]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001002	Male	No	0	Graduate	No	5849
1	LP001003	Male	Yes	1	Graduate	No	4583
2	LP001005	Male	Yes	0	Graduate	Yes	3000
3	LP001006	Male	Yes	0	Not Graduate	No	2583
4	LP001008	Male	No	0	Graduate	No	6000

In [6]: `df.tail()`

Out[6]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
609	LP002978	Female	No	0	Graduate	No	29
610	LP002979	Male	Yes	3+	Graduate	No	4
611	LP002983	Male	Yes	1	Graduate	No	80
612	LP002984	Male	Yes	2	Graduate	No	7
613	LP002990	Female	No	0	Graduate	Yes	4

In [7]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               614 non-null   object
1   Gender                601 non-null   object
2   Married               611 non-null   object
3   Dependents            599 non-null   object
4   Education             614 non-null   object
5   Self_Employed         582 non-null   object
6   ApplicantIncome       614 non-null   int64
7   CoapplicantIncome     614 non-null   float64
8   LoanAmount            592 non-null   float64
9   Loan_Amount_Term      600 non-null   float64
10  Credit_History        564 non-null   float64
11  Property_Area         614 non-null   object
12  Loan_Status           614 non-null   object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

In [8]: `df.columns`

Out[8]: Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'], dtype='object')

```
In [9]: df['Loan_ID'].nunique()
```

```
Out[9]: 614
```

```
In [10]: df.drop(columns = ['Loan_ID'],inplace = True)
df
```

```
Out[10]:
```

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome
0	Male	No	0	Graduate	No	5849	
1	Male	Yes	1	Graduate	No	4583	
2	Male	Yes	0	Graduate	Yes	3000	
3	Male	Yes	0	Not Graduate	No	2583	
4	Male	No	0	Graduate	No	6000	
...	...	...	...	...	...	...	...
609	Female	No	0	Graduate	No	2900	
610	Male	Yes	3+	Graduate	No	4106	
611	Male	Yes	1	Graduate	No	8072	
612	Male	Yes	2	Graduate	No	7583	
613	Female	No	0	Graduate	Yes	4583	

614 rows × 12 columns



```
In [11]: df['Gender'].unique()
```

```
Out[11]: array(['Male', 'Female', nan], dtype=object)
```

```
In [12]: df['Gender'].value_counts()
```

```
Out[12]: Gender
Male      489
Female    112
Name: count, dtype: int64
```

```
In [13]: df[df['Gender']=='Male']
```

Out[13]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome
0	Male	No	0	Graduate	No	5849	
1	Male	Yes	1	Graduate	No	4583	
2	Male	Yes	0	Graduate	Yes	3000	
3	Male	Yes	0	Not Graduate	No	2583	
4	Male	No	0	Graduate	No	6000	
...	...	...	...	...	...	...	...
607	Male	Yes	2	Not Graduate	No	3987	
608	Male	Yes	0	Graduate	No	3232	
610	Male	Yes	3+	Graduate	No	4106	
611	Male	Yes	1	Graduate	No	8072	
612	Male	Yes	2	Graduate	No	7583	

489 rows × 12 columns



In [14]: `df['Married'].unique`

Out[14]: <bound method Series.unique of 0 No
 1 Yes
 2 Yes
 3 Yes
 4 No
 ...
 609 No
 610 Yes
 611 Yes
 612 Yes
 613 No
 Name: Married, Length: 614, dtype: object>

In [15]: `df['Dependents'].unique()`

Out[15]: array(['0', '1', '2', '3+', nan], dtype=object)

In [16]: `df['Dependents'].value_counts()`

Out[16]: Dependents
 0 345
 1 102
 2 101
 3+ 51
 Name: count, dtype: int64

In [17]: `df['Education'].unique()`

Out[17]: array(['Graduate', 'Not Graduate'], dtype=object)

```
In [18]: df['Education'].value_counts()
```

```
Out[18]: Education
Graduate      480
Not Graduate   134
Name: count, dtype: int64
```

```
In [19]: df['Self_Employed'].unique()
```

```
Out[19]: array(['No', 'Yes', nan], dtype=object)
```

```
In [20]: df['ApplicantIncome'].unique()
```

```
Out[20]: array([ 5849, 4583, 3000, 2583, 6000, 5417, 2333, 3036, 4006,
12841, 3200, 2500, 3073, 1853, 1299, 4950, 3596, 3510,
4887, 2600, 7660, 5955, 3365, 3717, 9560, 2799, 4226,
1442, 3750, 4166, 3167, 4692, 3500, 12500, 2275, 1828,
3667, 3748, 3600, 1800, 2400, 3941, 4695, 3410, 5649,
5821, 2645, 4000, 1928, 3086, 4230, 4616, 11500, 2708,
2132, 3366, 8080, 3357, 3029, 2609, 4945, 5726, 10750,
7100, 4300, 3208, 1875, 4755, 5266, 1000, 3333, 3846,
2395, 1378, 3988, 2366, 8566, 5695, 2958, 6250, 3273,
4133, 3620, 6782, 2484, 1977, 4188, 1759, 4288, 4843,
13650, 4652, 3816, 3052, 11417, 7333, 3800, 2071, 5316,
2929, 3572, 7451, 5050, 14583, 2214, 5568, 10408, 5667,
2137, 2957, 3692, 23803, 3865, 10513, 6080, 20166, 2014,
2718, 3459, 4895, 3316, 14999, 4200, 5042, 6950, 2698,
11757, 2330, 14866, 1538, 10000, 4860, 6277, 2577, 9166,
2281, 3254, 39999, 9538, 2980, 1863, 7933, 3089, 4167,
9323, 3707, 2439, 2237, 8000, 1820, 51763, 3522, 5708,
4344, 3497, 2045, 5516, 6400, 1916, 4600, 33846, 3625,
39147, 2178, 2383, 674, 9328, 4885, 12000, 6033, 3858,
4191, 3125, 8333, 1907, 3416, 11000, 4923, 3992, 3917,
4408, 3244, 3975, 2479, 3418, 3430, 7787, 5703, 3173,
3850, 150, 3727, 5000, 4283, 2221, 4009, 2971, 7578,
3250, 4735, 4758, 2491, 3716, 3189, 3155, 5500, 5746,
3463, 3812, 3315, 5819, 2510, 2965, 3406, 6050, 9703,
6608, 2882, 1809, 1668, 3427, 2661, 16250, 3083, 6045,
5250, 14683, 4931, 6083, 2060, 3481, 7200, 5166, 4095,
4708, 4333, 2876, 3237, 11146, 2833, 2620, 3900, 2750,
3993, 3103, 4100, 4053, 3927, 2301, 1811, 20667, 3158,
3704, 4124, 9508, 3075, 4400, 3153, 4416, 6875, 4666,
2875, 1625, 2000, 3762, 20233, 7667, 2917, 2927, 2507,
2473, 3399, 2058, 3541, 4342, 3601, 3166, 15000, 8666,
4917, 5818, 4384, 2935, 63337, 9833, 5503, 1830, 4160,
2647, 2378, 4554, 2499, 3523, 6333, 2625, 9083, 8750,
2666, 2423, 3813, 3875, 5167, 4723, 4750, 3013, 6822,
6216, 5124, 6325, 19730, 15759, 5185, 3062, 2764, 4817,
4310, 3069, 5391, 5941, 7167, 4566, 2346, 3010, 5488,
9167, 9504, 1993, 3100, 3276, 3180, 3033, 3902, 1500,
2889, 2755, 1963, 7441, 4547, 2167, 2213, 8300, 81000,
3867, 6256, 6096, 2253, 2149, 2995, 1600, 1025, 3246,
5829, 2720, 7250, 14880, 4606, 5935, 2920, 2717, 8624,
6500, 12876, 2425, 10047, 1926, 10416, 7142, 3660, 7901,
4707, 37719, 3466, 3539, 3340, 2769, 2309, 1958, 3948,
2483, 7085, 3859, 4301, 3708, 4354, 8334, 2083, 7740,
3015, 5191, 2947, 16692, 210, 3450, 2653, 4691, 5532,
16525, 6700, 2873, 16667, 4350, 3095, 10833, 3547, 18333,
2435, 2699, 5333, 3691, 17263, 3597, 3326, 4625, 2895,
6283, 645, 3159, 4865, 4050, 3814, 20833, 3583, 13262,
3598, 6065, 3283, 2130, 5815, 2031, 3074, 4683, 3400,
2192, 5677, 7948, 4680, 17500, 3775, 5285, 2679, 6783,
4281, 3588, 11250, 18165, 2550, 6133, 3617, 6417, 4608,
2138, 3652, 2239, 3017, 2768, 3358, 2526, 2785, 6633,
2492, 2454, 3593, 5468, 2667, 10139, 3887, 4180, 3675,
19484, 5923, 5800, 8799, 4467, 3417, 5116, 16666, 6125,
6406, 3087, 3229, 1782, 3182, 6540, 1836, 1880, 2787,
2297, 2165, 2726, 9357, 16120, 3833, 6383, 2987, 9963,
5780, 416, 2894, 3676, 3987, 3232, 2900, 4106, 8072,
7583]), dtype=int64)
```

```
In [21]: df['CoapplicantIncome'].unique()
```

```
Out[21]: array([0.00000000e+00, 1.50800000e+03, 2.35800000e+03, 4.19600000e+03,
1.51600000e+03, 2.50400000e+03, 1.52600000e+03, 1.09680000e+04,
7.00000000e+02, 1.84000000e+03, 8.10600000e+03, 2.84000000e+03,
1.08600000e+03, 3.50000000e+03, 5.62500000e+03, 1.91100000e+03,
1.91700000e+03, 2.92500000e+03, 2.25300000e+03, 1.04000000e+03,
2.08300000e+03, 3.36900000e+03, 1.66700000e+03, 3.00000000e+03,
2.06700000e+03, 1.33000000e+03, 1.45900000e+03, 7.21000000e+03,
1.66800000e+03, 1.21300000e+03, 2.33600000e+03, 3.44000000e+03,
2.27500000e+03, 1.64400000e+03, 1.16700000e+03, 1.59100000e+03,
2.20000000e+03, 2.25000000e+03, 2.85900000e+03, 3.79600000e+03,
3.44900000e+03, 4.59500000e+03, 2.25400000e+03, 3.06600000e+03,
1.87500000e+03, 1.77400000e+03, 4.75000000e+03, 3.02200000e+03,
4.00000000e+03, 2.16600000e+03, 1.88100000e+03, 2.53100000e+03,
2.00000000e+03, 2.11800000e+03, 4.16700000e+03, 2.90000000e+03,
5.65400000e+03, 1.82000000e+03, 2.30200000e+03, 9.97000000e+02,
3.54100000e+03, 3.26300000e+03, 3.80600000e+03, 3.58300000e+03,
7.54000000e+02, 1.03000000e+03, 1.12600000e+03, 3.60000000e+03,
2.33300000e+03, 4.11400000e+03, 2.28300000e+03, 1.39800000e+03,
2.14200000e+03, 2.66700000e+03, 8.98000000e+03, 2.01400000e+03,
1.64000000e+03, 3.85000000e+03, 2.56900000e+03, 1.92900000e+03,
7.75000000e+03, 1.43000000e+03, 2.03400000e+03, 4.48600000e+03,
1.42500000e+03, 1.66600000e+03, 8.30000000e+02, 3.75000000e+03,
1.04100000e+03, 1.28000000e+03, 1.44700000e+03, 3.16600000e+03,
3.33300000e+03, 1.76900000e+03, 7.36000000e+02, 1.96400000e+03,
1.61900000e+03, 1.13000000e+04, 1.45100000e+03, 7.25000000e+03,
5.06300000e+03, 2.13800000e+03, 5.29600000e+03, 2.58300000e+03,
2.36500000e+03, 2.81600000e+03, 2.50000000e+03, 1.08300000e+03,
1.25000000e+03, 3.02100000e+03, 9.83000000e+02, 1.80000000e+03,
1.77500000e+03, 2.38300000e+03, 1.71700000e+03, 2.79100000e+03,
1.01000000e+03, 1.69500000e+03, 2.05400000e+03, 2.59800000e+03,
1.77900000e+03, 1.26000000e+03, 5.00000000e+03, 1.98300000e+03,
5.70100000e+03, 1.30000000e+03, 4.41700000e+03, 4.33300000e+03,
1.84300000e+03, 1.86800000e+03, 3.89000000e+03, 2.16700000e+03,
7.10100000e+03, 2.10000000e+03, 4.25000000e+03, 2.20900000e+03,
3.44700000e+03, 1.38700000e+03, 1.81100000e+03, 1.56000000e+03,
1.85700000e+03, 2.22300000e+03, 1.84200000e+03, 3.27400000e+03,
2.42600000e+03, 8.00000000e+02, 9.85799988e+02, 3.05300000e+03,
2.41600000e+03, 3.33400000e+03, 2.54100000e+03, 2.93400000e+03,
1.75000000e+03, 1.80300000e+03, 1.86300000e+03, 2.40500000e+03,
2.13400000e+03, 1.89000000e+02, 1.59000000e+03, 2.98500000e+03,
4.98300000e+03, 2.16000000e+03, 2.45100000e+03, 1.79300000e+03,
1.83300000e+03, 4.49000000e+03, 6.88000000e+02, 4.60000000e+03,
1.58700000e+03, 1.22900000e+03, 2.33000000e+03, 2.45800000e+03,
3.23000000e+03, 2.16800000e+03, 4.58300000e+03, 6.25000000e+03,
5.05000000e+02, 3.16700000e+03, 3.66700000e+03, 3.03300000e+03,
5.26600000e+03, 7.87300000e+03, 1.98700000e+03, 9.23000000e+02,
4.99600000e+03, 4.23200000e+03, 1.60000000e+03, 3.13600000e+03,
2.41700000e+03, 2.11500000e+03, 1.62500000e+03, 1.40000000e+03,
4.84000000e+02, 2.00000000e+04, 2.40000000e+03, 2.03300000e+03,
3.23700000e+03, 2.77300000e+03, 1.41700000e+03, 1.71900000e+03,
4.30000000e+03, 1.61200008e+01, 2.34000000e+03, 1.85100000e+03,
1.12500000e+03, 5.06400000e+03, 1.99300000e+03, 8.33300000e+03,
1.21000000e+03, 1.37600000e+03, 1.71000000e+03, 1.54200000e+03,
1.25500000e+03, 1.45600000e+03, 1.73300000e+03, 2.46600000e+03,
4.08300000e+03, 2.18800000e+03, 1.66400000e+03, 2.91700000e+03,
2.07900000e+03, 1.50000000e+03, 4.64800000e+03, 1.01400000e+03,
1.87200000e+03, 1.60300000e+03, 3.15000000e+03, 2.43600000e+03,
2.78500000e+03, 1.13100000e+03, 2.15700000e+03, 9.13000000e+02,
1.70000000e+03, 2.85700000e+03, 4.41600000e+03, 3.68300000e+03,
5.62400000e+03, 5.30200000e+03, 1.48300000e+03, 6.66700000e+03,
```

```
3.01300000e+03, 1.28700000e+03, 2.00400000e+03, 2.03500000e+03,
6.66600000e+03, 3.66600000e+03, 3.42800000e+03, 1.63200000e+03,
1.91500000e+03, 1.74200000e+03, 1.42400000e+03, 7.16600000e+03,
2.08700000e+03, 1.30200000e+03, 5.50000000e+03, 2.04200000e+03,
3.90600000e+03, 5.36000000e+02, 2.84500000e+03, 2.52400000e+03,
6.63000000e+02, 1.95000000e+03, 1.78300000e+03, 2.01600000e+03,
2.37500000e+03, 3.25000000e+03, 4.26600000e+03, 1.03200000e+03,
2.66900000e+03, 2.30600000e+03, 2.42000000e+02, 2.06400000e+03,
4.61000000e+02, 2.21000000e+03, 2.73900000e+03, 2.23200000e+03,
3.38370000e+04, 1.52200000e+03, 3.41600000e+03, 3.30000000e+03,
1.00000000e+03, 4.16670000e+04, 2.79200000e+03, 4.30100000e+03,
3.80000000e+03, 1.41100000e+03, 2.40000000e+02])
```

```
In [22]: df['LoanAmount'].unique()
```

```
Out[22]: array([ nan, 128.,  66., 120., 141., 267.,  95., 158., 168., 349.,  70.,
109., 200., 114.,  17., 125., 100.,  76., 133., 115., 104., 315.,
116., 112., 151., 191., 122., 110.,  35., 201.,  74., 106., 320.,
144., 184.,  80.,  47.,  75., 134.,  96.,  88.,  44., 286.,  97.,
135., 180.,  99., 165., 258., 126., 312., 136., 172.,  81., 187.,
113., 176., 130., 111., 167., 265.,  50., 210., 175., 131., 188.,
 25., 137., 160., 225., 216.,  94., 139., 152., 118., 185., 154.,
 85., 259., 194.,  93., 370., 182., 650., 102., 290.,  84., 242.,
129.,  30., 244., 600., 255.,  98., 275., 121.,  63., 700.,  87.,
101., 495.,  67.,  73., 260., 108.,  58.,  48., 164., 170.,  83.,
 90., 166., 124.,  55.,  59., 127., 214., 240.,  72.,  60., 138.,
 42., 280., 140., 155., 123., 279., 192., 304., 330., 150., 207.,
436.,  78.,  54.,  89., 143., 105., 132., 480.,  56., 159., 300.,
376., 117.,  71., 490., 173.,  46., 228., 308., 236., 570., 380.,
296., 156., 103.,  45.,  65.,  53., 360.,  62., 218., 178., 239.,
405., 148., 190., 149., 153., 162., 230.,  86., 234., 246., 500.,
186., 119., 107., 209., 208., 243.,  40., 250., 311., 400., 161.,
196., 324., 157., 145., 181.,  26., 211.,  9., 205.,  36.,  61.,
146., 292., 142., 350., 496., 253.]])
```

```
In [23]: df['Loan_Amount_Term'].unique()
```

```
Out[23]: array([360., 120., 240.,  nan, 180.,  60., 300., 480.,  36.,  84., 12.]])
```

```
In [24]: df['Credit_History'].unique()
```

```
Out[24]: array([ 1.,  0., nan])
```

```
In [25]: df['Credit_History'] = df['Credit_History'].replace({1:'good', 0:'bad'})
```

```
In [26]: df['Credit_History'].unique()
```

```
Out[26]: array(['good', 'bad', nan], dtype=object)
```

```
In [27]: df['Property_Area'].unique()
```

```
Out[27]: array(['Urban', 'Rural', 'Semiurban'], dtype=object)
```

```
In [28]: df['Property_Area'].value_counts()
```



```
Out[28]: Property_Area
Semiurban    233
Urban        202
Rural        179
Name: count, dtype: int64
```

```
In [29]: df['Loan_Status'].unique()
```

```
Out[29]: array(['Y', 'N'], dtype=object)
```

```
In [30]: df['Loan_Status'].value_counts()
```

```
Out[30]: Loan_Status
Y      422
N      192
Name: count, dtype: int64
```

```
In [31]: continuous = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount'] # int or flo
discrete_categorical = ['Gender', 'Married', 'Education', 'Self_Employed', 'Cred
discrete_count = ['Dependents', 'Loan_Amount_Term'] # int only
```

Exploratory Data Analysis(EDA)

```
In [33]: df[continuous].describe()
```

```
Out[33]:
```

	ApplicantIncome	CoapplicantIncome	LoanAmount
count	614.000000	614.000000	592.000000
mean	5403.459283	1621.245798	146.412162
std	6109.041673	2926.248369	85.587325
min	150.000000	0.000000	9.000000
25%	2877.500000	0.000000	100.000000
50%	3812.500000	1188.500000	128.000000
75%	5795.000000	2297.250000	168.000000
max	81000.000000	41667.000000	700.000000

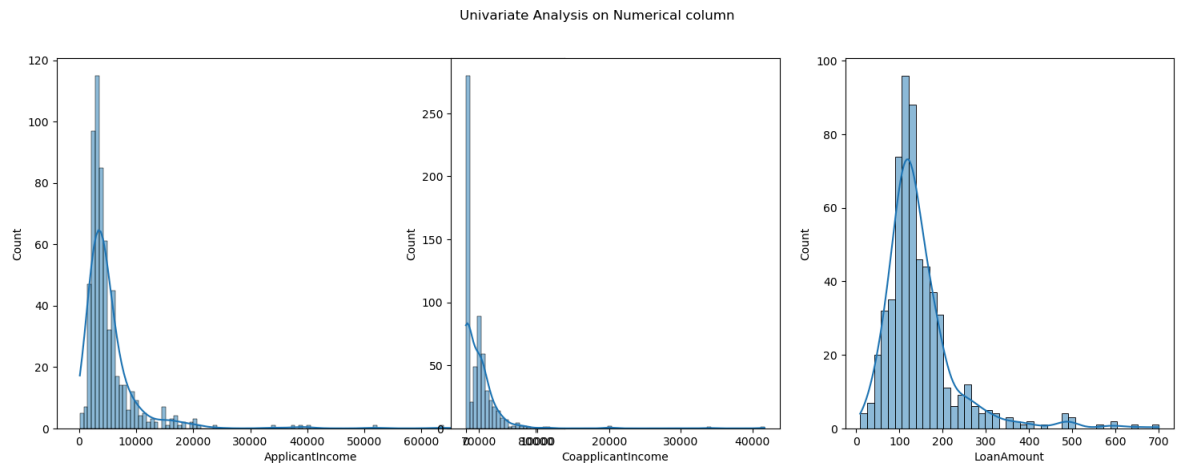
```
In [34]: plt.rcParams['figure.figsize'] = (18,6)

plt.subplot(1,2,1)
sns.histplot(df['ApplicantIncome'],kde = True) # total 3 subplots so 1= row, 3

plt.subplot(1,3,3)
sns.histplot(df['LoanAmount'],kde = True)

plt.subplot(1,3,2)
sns.histplot(df['CoapplicantIncome'], kde=True)

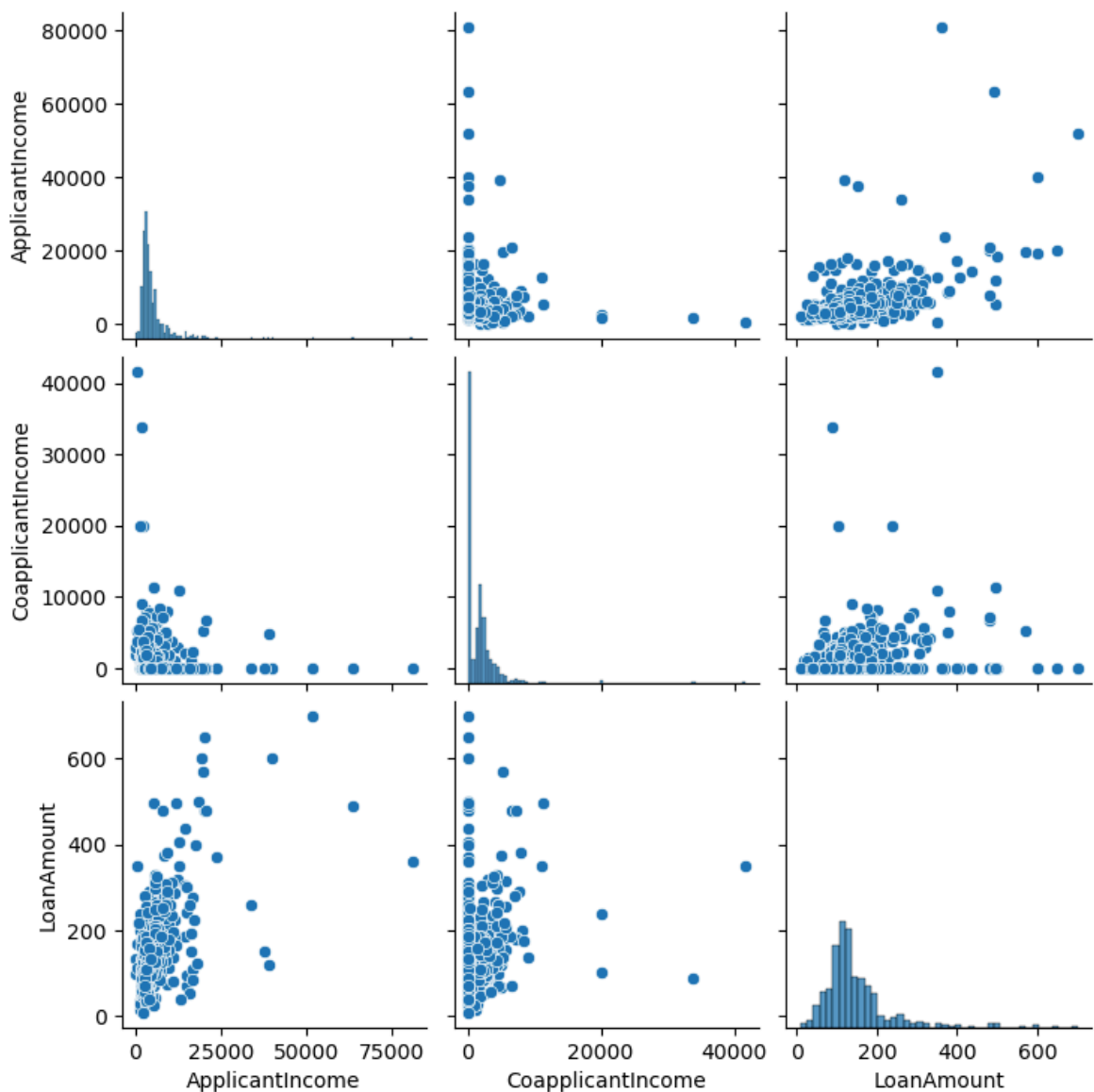
plt.suptitle('Univariate Analysis on Numerical column')
plt.show()
```



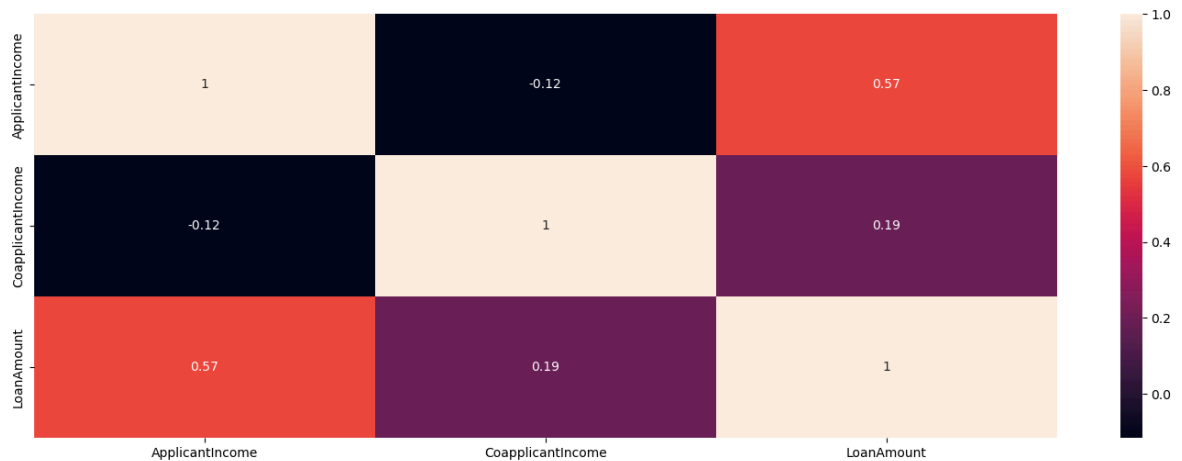
```
In [35]: df[continuous].skew()
```

```
Out[35]: ApplicantIncome    6.539513  
CoapplicantIncome    7.491531  
LoanAmount    2.677552  
dtype: float64
```

```
In [36]: sns.pairplot(df[continuous])  
plt.show()
```



```
In [37]: sns.heatmap(df[continuous].corr(),annot = True)
plt.show()
```



```
In [38]: df[discrete_categorical].describe()
```

```
Out[38]:
```

	Gender	Married	Education	Self_Employed	Credit_History	Property_Area	Loan_Status
count	601	611	614	582	564	614	614
unique	2	2	2	2	2	3	2
top	Male	Yes	Graduate	No	good	Semiurban	Completed
freq	489	398	480	500	475	233	489

```
In [39]: plt.rcParams['figure.figsize'] = (12,6)

plt.subplot(2,3,1)
sns.countplot(x='Gender', data=df)

plt.subplot(2,3,2)
sns.countplot(x='Married', data=df)

plt.subplot(2,3,3)
sns.countplot(x='Education', data=df)

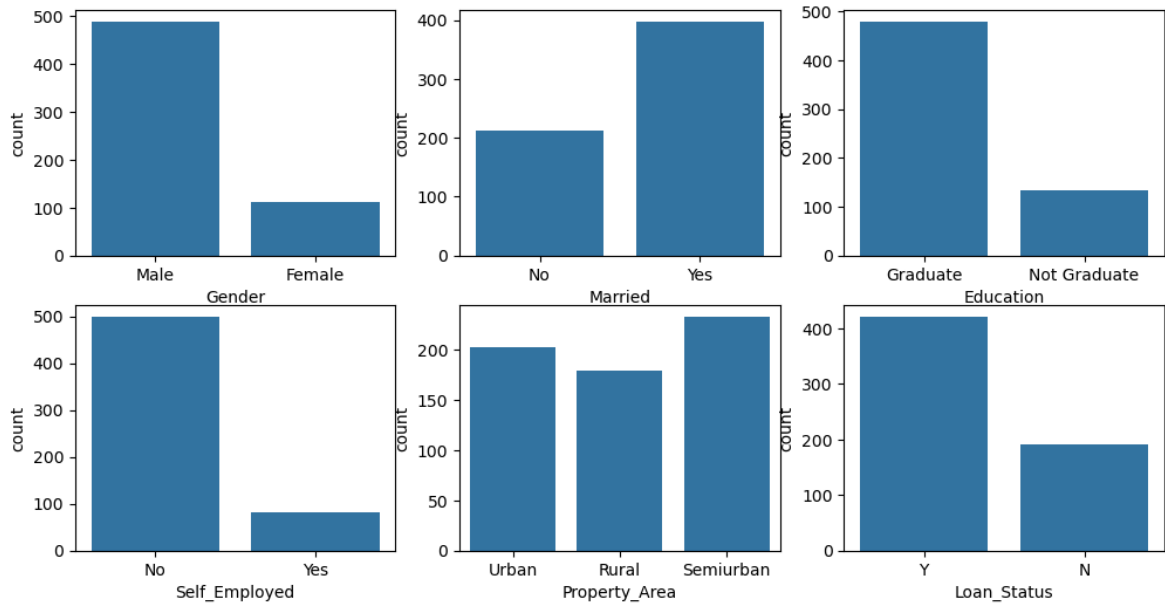
plt.subplot(2,3,4)
sns.countplot(x='Self_Employed', data=df)

plt.subplot(2,3,5)
sns.countplot(x='Property_Area', data=df)

plt.subplot(2,3,6)
sns.countplot(x='Loan_Status', data=df)

plt.suptitle('Univariate Analysis on Categorical Columns')
plt.show()
```

Univariate Analysis on Categorical Columns



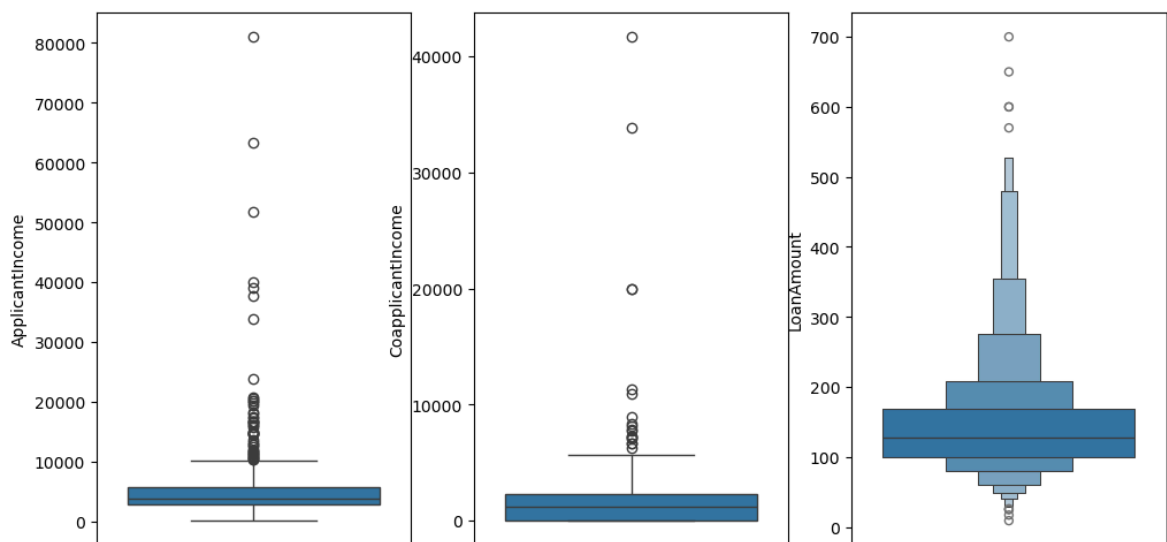
```
In [40]: # Outliers
plt.subplot(1,3,1)
sns.boxplot(df['ApplicantIncome'])

plt.subplot(1,3,2)
sns.boxplot(df['CoapplicantIncome'])

plt.subplot(1,3,3)
sns.boxenplot(df['LoanAmount'])

plt.suptitle('Outliers in the Data')
plt.show()
```

Outliers in the Data



Data Preparation

```
In [42]: df['Income'] = df['ApplicantIncome'] + df['CoapplicantIncome']
df.drop(columns = ['ApplicantIncome', 'CoapplicantIncome'], inplace = True)
df
```

Out[42]:

	Gender	Married	Dependents	Education	Self_Employed	LoanAmount	Loan_Amc
0	Male	No	0	Graduate	No	NaN	
1	Male	Yes	1	Graduate	No	128.0	
2	Male	Yes	0	Graduate	Yes	66.0	
3	Male	Yes	0	Not Graduate	No	120.0	
4	Male	No	0	Graduate	No	141.0	
...	...	...	...	...	...	...	...
609	Female	No	0	Graduate	No	71.0	
610	Male	Yes	3+	Graduate	No	40.0	
611	Male	Yes	1	Graduate	No	253.0	
612	Male	Yes	2	Graduate	No	187.0	
613	Female	No	0	Graduate	Yes	133.0	

614 rows × 11 columns



In [43]:

```
df['Dependents'] = df['Dependents'].replace({'3+':3})
df
df['Education'] = df['Education'].replace({'Not Graduate':'Non Graduate'})
df
```

Out[43]:

	Gender	Married	Dependents	Education	Self_Employed	LoanAmount	Loan_Amc
0	Male	No	0	Graduate	No	NaN	
1	Male	Yes	1	Graduate	No	128.0	
2	Male	Yes	0	Graduate	Yes	66.0	
3	Male	Yes	0	Non Graduate	No	120.0	
4	Male	No	0	Graduate	No	141.0	
...	...	...	...	...	...	...	...
609	Female	No	0	Graduate	No	71.0	
610	Male	Yes	3	Graduate	No	40.0	
611	Male	Yes	1	Graduate	No	253.0	
612	Male	Yes	2	Graduate	No	187.0	
613	Female	No	0	Graduate	Yes	133.0	

614 rows × 11 columns



```
In [44]: # Checking Missing Values:
df.isnull().sum()
```

```
Out[44]: Gender          13
Married              3
Dependents          15
Education            0
Self_Employed       32
LoanAmount           22
Loan_Amount_Term     14
Credit_History      50
Property_Area        0
Loan_Status          0
Income              0
dtype: int64
```

```
In [45]: df = df.dropna(subset = ['Income', 'LoanAmount', 'Loan_Amount_Term', 'Credit_Histor
df
```

```
Out[45]:
```

	Gender	Married	Dependents	Education	Self_Employed	LoanAmount	Loan_Amc
1	Male	Yes	1	Graduate	No	128.0	
2	Male	Yes	0	Graduate	Yes	66.0	
3	Male	Yes	0	Non Graduate	No	120.0	
4	Male	No	0	Graduate	No	141.0	
5	Male	Yes	2	Graduate	Yes	267.0	
...	...	...	...	...	...	...	
609	Female	No	0	Graduate	No	71.0	
610	Male	Yes	3	Graduate	No	40.0	
611	Male	Yes	1	Graduate	No	253.0	
612	Male	Yes	2	Graduate	No	187.0	
613	Female	No	0	Graduate	Yes	133.0	

529 rows × 11 columns



```
In [46]: df['Dependents'] = df['Dependents'].fillna(0)
```

```
In [47]: df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])
```

```
In [48]: df['Married'] = df['Married'].fillna(df['Married'].mode()[0])
```

```
In [49]: df['Self_Employed'] = df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])
```

```
In [50]: df.isnull().sum()
```

```
Out[50]: Gender          0
Married          0
Dependents       0
Education        0
Self_Employed    0
LoanAmount       0
Loan_Amount_Term 0
Credit_History   0
Property_Area     0
Loan_Status       0
Income           0
dtype: int64
```

```
In [51]: # Outliers Treatment
```

```
In [52]: # Encoding
df['Gender'] = df['Gender'].replace({'Male':1, 'Female':0}).astype('float')
df['Gender']
# ValueError: Cannot convert NA to integer occurs when attempting to cast a column
# i0When a column has NaN values, converting the column directly to an integer type
df['Married'] = df['Married'].replace({'Yes':1, 'No':0}).astype('float')
df['Education'] = df['Education'].replace({'Graduate':1, 'Non Graduate':0}).astype('float')
df['Self_Employed'] = df['Self_Employed'].replace({'Yes':1, 'No':0}).astype('float')
df['Property_Area'] = df['Property_Area'].replace({'Semiurban':1, 'Rural':0, 'Urban':2}).astype('float')
df['Credit_History'] = df['Credit_History'].replace({'good':1, 'bad':0}).astype('float')
df['Loan_Status'] = df['Loan_Status'].replace({'Y':1, 'N':0}).astype('float')
```

```
In [53]: df
```

```
Out[53]:
```

	Gender	Married	Dependents	Education	Self_Employed	LoanAmount	Loan_Amount_Term
1	1.0	1.0	1	1.0	0.0	128.0	
2	1.0	1.0	0	1.0	1.0	66.0	
3	1.0	1.0	0	0.0	0.0	120.0	
4	1.0	0.0	0	1.0	0.0	141.0	
5	1.0	1.0	2	1.0	1.0	267.0	
...	...	...	...	...	...	...	
609	0.0	0.0	0	1.0	0.0	71.0	
610	1.0	1.0	3	1.0	0.0	40.0	
611	1.0	1.0	1	1.0	0.0	253.0	
612	1.0	1.0	2	1.0	0.0	187.0	
613	0.0	0.0	0	1.0	1.0	133.0	

529 rows × 11 columns



```
In [54]: df['Dependents'] = df['Dependents'].astype('int')
df['Loan_Amount_Term'] = df['Loan_Amount_Term'].astype('int')
```

In [55]: df

Out[55]:

	Gender	Married	Dependents	Education	Self_Employed	LoanAmount	Loan_Amc
1	1.0	1.0	1	1.0	0.0	128.0	
2	1.0	1.0	0	1.0	1.0	66.0	
3	1.0	1.0	0	0.0	0.0	120.0	
4	1.0	0.0	0	1.0	0.0	141.0	
5	1.0	1.0	2	1.0	1.0	267.0	
...	...	...	...	...	...	...	...
609	0.0	0.0	0	1.0	0.0	71.0	
610	1.0	1.0	3	1.0	0.0	40.0	
611	1.0	1.0	1	1.0	0.0	253.0	
612	1.0	1.0	2	1.0	0.0	187.0	
613	0.0	0.0	0	1.0	1.0	133.0	

529 rows × 11 columns



In [56]: *# Transformations*
df[['Income', 'LoanAmount']].skew()

Out[56]: Income 5.777628
LoanAmount 2.607945
dtype: float64

In [57]: from scipy.stats import boxcox
df['Income'], a = boxcox(df['Income'])
df['LoanAmount'], c = boxcox(df['LoanAmount'])

In [58]: df[['Income', 'LoanAmount']].skew()

Out[58]: Income -0.027769
LoanAmount 0.038289
dtype: float64

In [59]: df['Loan_Amount_Term'] = df['Loan_Amount_Term']/12
df

Out[59]:

	Gender	Married	Dependents	Education	Self_Employed	LoanAmount	Loan_Amc
1	1.0	1.0	1	1.0	0.0	6.243267	
2	1.0	1.0	0	1.0	1.0	5.202610	
3	1.0	1.0	0	0.0	0.0	6.138816	
4	1.0	0.0	0	1.0	0.0	6.401083	
5	1.0	1.0	2	1.0	1.0	7.481931	
...	...	...	...	...	...	...	...
609	0.0	0.0	0	1.0	0.0	5.313992	
610	1.0	1.0	3	1.0	0.0	4.460306	
611	1.0	1.0	1	1.0	0.0	7.388071	
612	1.0	1.0	2	1.0	0.0	6.870559	
613	0.0	0.0	0	1.0	1.0	6.305603	

529 rows × 11 columns



```
In [60]: X = df.drop('Loan_Status',axis = 1)
y = df['Loan_Status']
```

```
In [61]: # Identifying best random state number
Train = []
Test = []
CV = []

for i in range(0,101):
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(X,y,test_size= 0.2, rand

    from sklearn.linear_model import LogisticRegression
    log_default = LogisticRegression()
    log_default.fit(X_train,y_train)

    ypred_train = log_default.predict(X_train)
    ypred_test = log_default.predict(X_test)

    from sklearn.metrics import accuracy_score
    Train.append(accuracy_score(y_train, ypred_train))
    Test.append(accuracy_score(y_test,ypred_test))

    from sklearn.model_selection import cross_val_score
    cv_scores = cross_val_score(log_default, X_train, y_train, cv=5, scoring='ac
    CV.append(cv_scores.mean())

em = pd.DataFrame({'Train':Train,'Test':Test,'CV':CV})
gm = em[(abs(em['Train'])-em['Test']<=0.05) & (abs(em['Test'])-em['CV']<=0.05))]
rs = gm[gm['CV']==gm['CV'].max()].index.to_list()[0]
print('best random_state number:',rs)
```

best random_state number: 70

```
In [121... # train_test_split
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.2,random_stat
```

Machine Learning Modelling & Evaluation

1. Logistic Regression

```
In [124... from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression()
log_model.fit(X_train,y_train) # Fit the model

ypred_train = log_model.predict(X_train) #Predict the model in X_train
ypred_test = log_model.predict(X_test) #Predict the model in X_test

print('Train Accuracy:', accuracy_score(y_train,ypred_train))
print('Cross validation score:', cross_val_score(log_model,X_train,y_train,cv =
print('Test Accuracy:', accuracy_score(y_test,ypred_test))
```

Train Accuracy: 0.8226950354609929

Cross validation score: [0.82352941 0.8 0.81176471 0.83333333 0.8452381]

Test Accuracy: 0.7830188679245284

2. KNN

```
In [128... from sklearn.neighbors import KNeighborsClassifier

estimator = KNeighborsClassifier()
param_grid = {'n_neighbors' : list(range(1,50))}

from sklearn.model_selection import GridSearchCV
knn_grid = GridSearchCV(estimator, param_grid, scoring = 'accuracy',cv = 5)
knn_grid.fit(X_train,y_train)

knn_model = knn_grid.best_estimator_

ypred_train = knn_model.predict(X_train)
ypred_test = knn_model.predict(X_test)

print('Train Accuracy:', accuracy_score(y_train, ypred_train))
print('Cross validation score:',cross_val_score(knn_model,X_train,y_train,cv = 5
print('Test Accuracy:', accuracy_score(y_test,ypred_test))
```

Train Accuracy: 0.7541371158392435

Cross validation score: [0.72941176 0.74117647 0.74117647 0.73809524 0.73809524]

Test Accuracy: 0.7075471698113207

```
In [132... knn_model = knn_grid.best_estimator_
knn_model
```

```
Out[132... KNeighborsClassifier
KNeighborsClassifier(n_neighbors=13)
```

3. Support Vector Machine(SVM)

```
In [140... from sklearn.svm import SVC
estimator = SVC()
param_grid = {'C':[0.01,0.1,1],'kernel':['linear','rbf','sigmoid','poly']}

from sklearn.model_selection import GridSearchCV
svm_grid = GridSearchCV(estimator, param_grid,scoring = 'accuracy',cv = 5)
svm_grid.fit(X_train,y_train)

svm_model = svm_grid.best_estimator_

ypred_train = svm_model.predict(X_train)
ypred_test = svm_model.predict(X_test)

print('Train Accuracy:',accuracy_score(y_train,ypred_train))
print('Cross validation score:',cross_val_score(svm_model,X_train,y_train,cv = 5)
print('Test Accuracy:',accuracy_score(y_test,ypred_test))
```

Train Accuracy: 0.8226950354609929

Cross validation score: [0.82352941 0.8 0.81176471 0.83333333 0.8452381]

Test Accuracy: 0.7830188679245284

```
In [142... svm_model = svm_grid.best_estimator_
svm_model
```

```
Out[142... SVC
SVC(C=0.1, kernel='linear')
```

4. Decision Tree Classifier

```
In [145... from sklearn.tree import DecisionTreeClassifier
estimator = DecisionTreeClassifier(random_state = rs)
param_grid = {'criterion':['gini','entropy'],'max_depth':list(range(1,16))}

from sklearn.model_selection import GridSearchCV
dt_grid = GridSearchCV(estimator, param_grid,scoring = 'accuracy',cv = 5)
dt_grid.fit(X_train,y_train)

dt = dt_grid.best_estimator_
dt_fi = dt.feature_importances_

index = [i for i,x in enumerate(dt_fi) if x>0]

X_train_dt = X_train.iloc[:,index]
X_test_dt = X_test.iloc[:,index]

dt.fit(X_train_dt,y_train)

ypred_train = dt.predict(X_train_dt)
ypred_test = dt.predict(X_test_dt)

print('Train Accuracy:',accuracy_score(y_train,ypred_train))
print('Cross validation score:',cross_val_score(dt,X_train,y_train,cv = 5, scoring = 'accuracy'))
print('Test Accuracy:',accuracy_score(y_test,ypred_test))
```

Train Accuracy: 0.8226950354609929

Cross validation score: [0.82352941 0.8 0.81176471 0.83333333 0.8452381]

Test Accuracy: 0.7830188679245284

5. Random Forest Classifier

In [149...

```

from sklearn.ensemble import RandomForestClassifier
estimator = RandomForestClassifier(random_state = rs)
param_grid = {'n_estimators':list(range(1,51))}

from sklearn.model_selection import GridSearchCV
rf_grid = GridSearchCV(estimator, param_grid,scoring = 'accuracy',cv = 5)
rf_grid.fit(X_train,y_train)

rf = rf_grid.best_estimator_
rf_fi = rf.feature_importances_

index = [i for i,x in enumerate(rf_fi) if x>0]

X_train_rf = X_train.iloc[:,index]
X_test_rf = X_test.iloc[:,index]

rf.fit(X_train_rf,y_train)

ypred_train = rf.predict(X_train_rf)
ypred_test = rf.predict(X_test_rf)

print('Train Accuracy:',accuracy_score(y_train,ypred_train))
print('Cross validation score:',cross_val_score(rf,X_train,y_train,cv = 5, scoring=
print('Test Accuracy:',accuracy_score(y_test,ypred_test))

```

Train Accuracy: 0.9905437352245863

Cross validation score: [0.77647059 0.76470588 0.81176471 0.83333333 0.8452381]

Test Accuracy: 0.7641509433962265

7. GradientBoost Classifier

In [152...

```

from sklearn.ensemble import GradientBoostingClassifier
estimator = GradientBoostingClassifier(random_state = rs)
param_grid = {'n_estimators':list(range(1,50)), 'learning_rate':[0.1,0.2,0.3,0.4,

from sklearn.model_selection import GridSearchCV
gb_grid = GridSearchCV(estimator, param_grid,scoring = 'accuracy',cv = 5)
gb_grid.fit(X_train,y_train)

gb = gb_grid.best_estimator_
gb_fi = gb.feature_importances_

index = [i for i,x in enumerate(gb_fi) if x>0]

X_train_gb = X_train.iloc[:,index]
X_test_gb = X_test.iloc[:,index]

gb.fit(X_train_gb,y_train)

ypred_train = gb.predict(X_train_gb)
ypred_test = gb.predict(X_test_gb)

print('Train Accuracy:',accuracy_score(y_train,ypred_train))
print('Cross validation score:',cross_val_score(gb,X_train,y_train,cv = 5, scoring=
print('Test Accuracy:',accuracy_score(y_test,ypred_test))

```

Train Accuracy: 0.8392434988179669

Cross validation score: [0.82352941 0.78823529 0.82352941 0.8452381 0.8452381]

Test Accuracy: 0.7735849056603774

8.XGBoost Classifier

In [154...

```
from xgboost import XGBClassifier
estimator = XGBClassifier(random_state = rs)
param_grid = {'n_estimators':[10,20,40,100], 'max_depth':[3,4,5], 'gamma':[0,0.15]

from sklearn.model_selection import GridSearchCV
xgb_grid = GridSearchCV(estimator, param_grid,scoring = 'accuracy',cv = 5)
xgb_grid.fit(X_train,y_train)

xgb = xgb_grid.best_estimator_
xgb_fi = xgb.feature_importances_

index = [i for i,x in enumerate(xgb_fi) if x>0]

X_train_xgb = X_train.iloc[:,index]
X_test_xgb = X_test.iloc[:,index]

xgb.fit(X_train_xgb,y_train)

ypred_train = xgb.predict(X_train_xgb)
ypred_test = xgb.predict(X_test_xgb)

print('Train Accuracy:',accuracy_score(y_train,ypred_train))
print('Cross validation score:',cross_val_score(xgb,X_train,y_train,cv = 5, scor
print('Test Accuracy:',accuracy_score(y_test,ypred_test))
```

Train Accuracy: 0.8557919621749409

Cross validation score: [0.83529412 0.8 0.82352941 0.83333333 0.85714286]

Test Accuracy: 0.7735849056603774

Completed

In []: